

FYP2 Final Report

Project title: Automated Manuscript Template Compliance Checker for Academic Journals

Student ID: 241UC240J2

Student name: GOH PEI CHUNG

Programme: Bachelor of Computer Science (Data Science)

Supervisor: Prof. Haw Su Cheng

FYP type: Research-Based with proof-of-concept prototype implementation

Validated case study: Journal of Informatics and Web Engineering (JIWE)

Preliminary Pages

Cover Page

FINAL YEAR PROJECT FINAL REPORT

Automated Manuscript Template Compliance Checker for Academic Journals

241UC240J2

GOH PEI CHUNG

BACHELOR OF COMPUTER SCIENCE (DATA SCIENCE)

June 2026

Title Page

Automated Manuscript Template Compliance Checker for Academic Journals

BY

241UC240J2 GOH PEI CHUNG

**PROJECT FINAL REPORT SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIREMENT FOR THE DEGREE OF BACHELOR OF COMPUTER
SCIENCE (DATA SCIENCE)**

in the Faculty of Computing and Informatics

MULTIMEDIA UNIVERSITY

MALAYSIA

June 2026

Copyright Page

Copyright of this report belongs to Universiti Telekom Sdn. Bhd. as qualified by the applicable Multimedia University intellectual property policy. No part of this publication may be reproduced, stored, or transmitted without proper permission and acknowledgement.

Declaration

I hereby declare that the work contained in this report has been completed by myself and that no portion of this work has been submitted in support of any application for any other degree or qualification of this or any other university or institution of learning.

Name of candidate: GOH PEI CHUNG

Faculty of Computing and Informatics

Multimedia University

Date: 29 June 2026

Acknowledgement

I would like to express my sincere appreciation to my supervisor, Prof. Haw Su Cheng, for her guidance, feedback, and encouragement throughout the development of this Final Year Project. Her advice helped me refine the project from an initial hybrid AI-rules prototype into a more practical, rule-first, and evaluable research prototype.

I would also like to thank the Faculty of Computing and Informatics, Multimedia University, for providing the academic guidance and project structure required to complete this work. I am grateful to the lecturers, reviewers, and peers who provided comments during FYP1 and FYP2, especially on the importance of practical evaluation, explainability, and realistic system limitations.

Finally, I would like to thank my family and friends for their support throughout the project period.

Abstract

Academic journals commonly reject manuscripts at the initial screening stage when submissions do not follow the required formatting template. These desk rejections can delay publication and increase manual checking workload for authors, editors, and academic staff. This project proposes a rule-first, profile-based manuscript template compliance checking approach for academic journals. A proof-of-concept web prototype

was implemented using the Journal of Informatics and Web Engineering (JIWE) as the main validated case study. The system extracts or loads journal template rules, classifies manuscript paragraphs using deterministic profile-aware rules, checks objective formatting and structural requirements, generates categorized issues, and applies conservative auto-corrections where deterministic correction is safe. Optional AI support is limited to explaining rule-detected issues and generating review guidance; it does not make compliance decisions or approve manuscripts. Evaluation was conducted using smoke testing and auto-fix before/after validation on published and flawed JIWE samples. The published sample run checked 10 documents and reduced detected issues from 88 to 14, while the declined or flawed sample run checked 6 documents and reduced detected issues from 98 to 28. All evaluated auto-fix runs were marked safe by post-fix validation. The results show that the proposed approach can support initial manuscript screening and reduce common formatting issues, while remaining transparent about limitations such as PDF fidelity, manual review needs, and the requirement for labelled precision, recall, and F1-score evaluation.

Keywords: manuscript formatting, template compliance, rule-based validation, academic journal, auto-correction, document processing

Table of Contents

Generate using Microsoft Word after final formatting.

List of Tables

Table 2.1: Comparison of Existing Tools and Proposed Approach

Table 4.1: JIWE Rule Categories and Examples

Table 5.1: Test Document Groups

Table 5.2: Published Sample Auto-Fix Before and After Results

Table 5.3: Declined or Flawed Sample Auto-Fix Before and After Results

Table 5.4: Recommended Subject Expert Evaluation Form

List of Figures

Figure 3.1: Rule-First Profile-Based Compliance Framework

Figure 4.1: Overall Prototype Architecture

Figure 4.2: Manuscript Checking Workflow

Figure 4.3: Auto-Fix and Post-Fix Validation Workflow

Figure 5.1: Streamlit Upload Interface

Figure 5.2: Issue Category Results Interface

Figure 5.3: Download and Report Output Interface

List of Abbreviations

AI: Artificial Intelligence

API: Application Programming Interface

DOCX: Microsoft Word Open XML Document

F1: Harmonic mean of precision and recall

FYP: Final Year Project

JIWE: Journal of Informatics and Web Engineering

LLM: Large Language Model

OCR: Optical Character Recognition

PDF: Portable Document Format

UI: User Interface

List of Appendices

Appendix A: Updated Gantt Chart

Appendix B: FYP2 Meeting Logs

Appendix C: Turnitin Similarity Index Page

Appendix D: Evaluation Result Samples

Appendix E: Subject Expert Evaluation Form

Appendix F: Selected Prototype Screenshots

Appendix G: Code Repository Summary

Chapter 1: Introduction

1.1 Background of the Research

Academic journals provide manuscript templates to standardize submissions before peer review and publication. These templates normally define page layout, title format, author information, abstract structure, keywords, headings, figure captions, table captions, references, declarations, and other formatting requirements. When authors do not follow these requirements, editors or administrative staff must spend additional time checking the document, requesting corrections, or rejecting the manuscript before it reaches full review.

Formatting-related desk rejection is a practical problem because it does not necessarily reflect the scientific quality of a manuscript. A paper may contain useful research but still fail the initial screening because of incorrect margins, missing sections, inconsistent fonts, wrong caption placement, missing declarations, or reference formatting issues. Manual checking is also repetitive and can be inconsistent when different reviewers focus on different formatting details.

This research focuses on academic manuscript template compliance checking as a document processing problem. Instead of using an LLM as the main decision maker, the project proposes a rule-first and profile-based approach. Objective formatting requirements are represented as deterministic rules, while journal-specific differences are stored in journal profiles. A working prototype was developed to demonstrate this approach using JIWE as the primary validated journal profile.

1.2 Problem Statement

Academic manuscript template compliance checking is still commonly performed manually. This creates several problems. First, the process is time-consuming because editors, lecturers, or authors must inspect many formatting details across long documents. Second, manual checking can be inconsistent because some issues may be missed or interpreted differently by different reviewers. Third, many existing general writing tools focus on grammar, spelling, or citation management rather than journal-specific formatting compliance. Fourth, AI-based approaches can be too expensive or unreliable if they are used as the main compliance decision maker, especially for objective formatting rules that can be checked deterministically.

Therefore, there is a need for a transparent and practical method that can check objective manuscript formatting requirements, identify common template violations, apply safe corrections where possible, and present remaining issues for manual review.

1.3 Research Purpose

The purpose of this research is to design, implement, and evaluate a rule-first, profile-based manuscript template compliance checking approach for academic journals. The project aims to demonstrate that deterministic rules and journal profiles can support explainable issue detection and safer auto-correction for common manuscript formatting problems.

1.4 Research Objectives

The objectives of this research are:

1. To design a profile-based rule representation for academic journal manuscript templates.
2. To implement a rule-based manuscript compliance checker using JIWE as the primary validated journal profile.
3. To develop safer auto-correction for supported formatting issues while preserving risky content and layout elements for manual review.
4. To generate user-facing outputs including categorized issues, corrected DOCX files, highlighted original DOCX files, and comparison reports.
5. To evaluate the prototype using smoke testing, auto-fix before/after validation, and a recommended manual-label and subject expert evaluation framework.

1.5 Research Questions

The research questions are:

6. How can journal-specific manuscript template requirements be represented in a profile-based rule structure?
7. How effectively can deterministic rules detect common formatting and structural compliance issues in academic manuscripts?
8. Which formatting issues can be safely corrected automatically without damaging manuscript content or layout?
9. How can optional AI explanations be used without making AI responsible for compliance decisions?
10. How useful is the prototype for supporting initial manuscript screening by authors, lecturers, or editors?

1.6 Project Scope

The project scope includes the design and implementation of a proof-of-concept prototype for academic manuscript template compliance checking. The main validated case study is JIWE. The prototype supports DOCX template upload, default JIWE profile rules, DOCX manuscript checking, issue categorisation, compliance index calculation,

safer auto-fix, highlighted output, comparison report generation, and optional AI explanation.

PDF upload is supported only as a convenience path when conversion dependencies are available. DOCX remains the primary reliable format because it preserves editable paragraph, run, style, margin, header, footer, and section metadata. PDF download is not guaranteed in every environment because server-side conversion without Microsoft Word can shift Word template layout.

The project does not perform research quality assessment, full citation database verification, complete multi-journal universal validation, full manuscript rewriting, or final editorial approval.

1.7 Significance of the Research

This research is significant because it addresses a practical workflow problem in academic publishing and university manuscript preparation. The proposed system can help authors detect formatting problems earlier, reduce repeated correction cycles, and help reviewers focus on issues that require human judgement. The rule-first design improves transparency because each issue is linked to an expected formatting rule rather than a black-box AI decision.

The project also contributes a practical architecture for combining deterministic rule-based checking with optional AI explanation. This separation is important because formatting compliance should be objective, auditable, and reproducible, while natural language explanation can improve user understanding.

1.8 Organization of Chapters

Chapter 1 introduces the research background, problem statement, objectives, research questions, scope, and significance. Chapter 2 reviews literature related to manuscript formatting, document layout analysis, rule-based validation, AI-assisted document understanding, and profile-based processing. Chapter 3 presents the theoretical framework for template compliance checking. Chapter 4 describes the research methodology and proposed prototype. Chapter 5 presents implementation evidence, evaluation data, and results. Chapter 6 discusses the findings, strengths, limitations, and implications. Chapter 7 concludes the project and provides recommendations for future work.

1.9 Summary

This chapter introduced the need for a rule-first manuscript template compliance checker. The project is framed as a research-based proof-of-concept prototype that uses JIWE as a validated case study and optional AI only for explanation support.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the background concepts needed to understand academic manuscript template compliance checking. The review covers academic formatting and editorial screening, document layout analysis, rule-based document validation, AI-assisted document understanding, hybrid profile-based approaches, and a comparison with existing tools.

2.2 Academic Manuscript Formatting and Editorial Screening

Academic journals use templates to ensure that manuscripts follow a consistent publication style. Formatting requirements can include page size, margins, title format, author information, abstract word count, keyword formatting, headings, captions, reference style, declarations, and page layout. These requirements allow editorial staff to process submissions more efficiently and maintain consistency across published articles.

However, template compliance is often checked manually. Manual checking requires reviewers to inspect both content structure and visual formatting. This can be tedious when documents are long or when formatting errors appear across many paragraphs. Desk rejection due to formatting problems can delay submission even when the manuscript's research contribution may be valid.

2.3 Document Layout Analysis

Document layout analysis studies how document elements such as titles, headings, paragraphs, tables, figures, headers, footers, and references can be identified and interpreted. Research such as PubLayNet (Zhong et al., 2019), LayoutLM (Xu et al., 2020), and DocLayNet (Pfitzmann et al., 2022) shows that document structure can be modelled using layout and text signals. These works are important because manuscript template checking depends on correctly identifying document regions before applying rules.

In editable documents such as DOCX files, layout information can be extracted from paragraph properties, run properties, section settings, and XML elements. Microsoft WordprocessingML represents Word documents as structured XML parts, making it possible to inspect paragraphs, runs, tables, headers, and sections programmatically (Microsoft, 2024). In fixed-layout documents such as PDF files, this process is more difficult because the file may not preserve editable Word structure. Bibliographic extraction tools such as GROBID (Lopez, 2009) also show the value and difficulty of extracting reliable structure from scholarly documents.

For this project, DOCX is treated as the primary processing format because it preserves structured formatting metadata. PDF is treated as a secondary input path only when conversion is possible.

2.4 Rule-Based Document Validation

Rule-based validation checks a document against predefined conditions. For manuscript formatting, examples include checking whether the title uses the expected font size, whether body text is justified, whether required sections exist, and whether references use the expected line spacing. Rule-based validation is suitable for objective requirements because it is explainable and reproducible.

The limitation of rule-based validation is that it needs accurate rule definitions and robust element classification. A paragraph must first be identified as a title, heading, body paragraph, caption, declaration, or reference before the correct rule can be applied.

2.5 AI-Assisted Document Understanding

AI methods can support document understanding by classifying document elements or generating explanations. Layout-aware models such as LayoutLM can learn text and layout relationships, but they require labelled data, training, and evaluation before they should be trusted for journal-specific compliance decisions. For this FYP prototype, using a large LLM as the main compliance checker would be inefficient and difficult to verify because objective formatting rules do not require generative reasoning when the required value is known. Therefore, this project limits AI to explanation and review guidance. The compliance decision remains deterministic.

2.6 Hybrid and Profile-Based Document Processing Approaches

A profile-based approach stores journal-specific rules separately from the checking logic. This reduces hard coding and allows the same checker architecture to be adapted to different journals by changing the profile. The current prototype uses JIWE as the validated profile and a generic profile as fallback. Extracted template rules can be merged with profile defaults to handle cases where some rules are not explicitly available from the uploaded template.

2.7 Existing Tools Comparison

General writing tools and word processors can detect grammar, spelling, and some style problems, but they usually do not validate manuscript formatting against a specific journal template. Citation managers help generate references but do not check all template layout requirements. PDF converters can transform file formats but may damage layout or lose editable structure.

Table 2.1 compares the proposed system with common existing approaches used during manuscript preparation and screening.

2.8 Related Work Summary Table

Table 2.1: Comparison of Existing Tools and the Proposed Approach

Area	Existing approach	Strength	Limitation	Relevance to this project
Manual editorial screening	Human reviewers inspect formatting and content	High judgement quality	Slow and inconsistent for repetitive formatting	Motivates automation
Word processors	Microsoft Word and similar tools	Widely available and preserve DOCX layout	Not journal-profile specific	Baseline preparation tool
Citation managers	Reference formatting tools	Useful for managing citations	Limited checking of full manuscript layout	Related but incomplete
Document layout analysis	Layout-aware models such as PubLayNet, LayoutLM, and DocLayNet	Can identify document regions using text and layout	Requires labelled data and may not preserve editable Word formatting	Informs future classifier work
Bibliographic extraction	Tools such as GROBID	Extracts scholarly metadata and references	Focused on bibliographic structure rather than full template compliance	Relevant to future reference checking
Rule-based validation	Deterministic checks against explicit rules	Explainable and reproducible	Depends on accurate rules and classification	Core method of this project
LLM assistance	Generative explanation or review guidance	User-friendly explanations	Not reliable as final decision maker for objective formatting	Optional explanation layer only

2.9 Research Gap

Existing tools do not provide a focused, explainable, and journal-profile-based method for checking manuscript template compliance and applying safe corrections. There is a gap between manual editorial checking and fully automated AI-based document

understanding. This project addresses the gap by proposing a rule-first system where journal rules are explicit, checking is deterministic, auto-fix is conservative, and AI is used only for explanations.

2.10 Summary

This chapter reviewed the need for automated manuscript template compliance checking and justified the use of a rule-first profile-based approach.

Chapter 3: Theoretical Framework

3.1 Introduction

This chapter presents the theoretical framework behind the proposed manuscript template compliance checker. The framework combines document template compliance theory, rule-based validation, profile-based journal rule representation, document element classification, compliance scoring, safer auto-correction, and optional AI explanation.

3.2 Document Template Compliance Theory

Document template compliance can be understood as the degree to which a manuscript matches the expected structure and formatting defined by a target template. A template requirement may be structural, such as the presence of an abstract or references section, or formatting-based, such as a font size, alignment, margin, or line spacing requirement.

3.3 Rule-Based Validation Theory

Rule-based validation compares extracted document features against expected values. Each rule includes a target element, a property, an expected value, and a method for comparing the actual value. For example, a body text paragraph may be checked for font name, font size, bold style, alignment, and line spacing.

3.4 Profile-Based Journal Rule Representation

The profile-based approach represents journal-specific requirements in external JSON profiles. In the current implementation, `template_profiles/jiwe.json` stores required sections, declarations, classification patterns, rule weights, and formatting rules. This allows the checker to separate journal-specific knowledge from general checking logic.

3.5 Document Element Classification

Before rules can be applied, paragraphs must be classified into roles such as journal header, paper title, author information, body text, heading, subheading, caption, declaration, or reference. The prototype uses deterministic classification based on text patterns, paragraph position, formatting, and profile-defined patterns.

3.6 Compliance Scoring Framework

The compliance index is a user-facing indicator calculated from detected issues and rule weights. It helps users understand the overall formatting condition of a manuscript. However, it is not the same as formal detection accuracy. Formal accuracy should be measured using precision, recall, and F1-score against manually labelled ground truth.

3.7 Safer Auto-Correction and Formatting Preservation Concept

Auto-correction can reduce repetitive formatting errors, but it can also damage layout if applied too broadly. Therefore, the prototype only applies deterministic fixes for

supported issue properties. Examples include font name, font size, bold, italic, alignment, line spacing, margins, and selected header spacing corrections. Risky tasks such as moving figures or generating missing content are left for manual review.

3.8 Optional AI Explanation Layer

The optional AI layer is not part of the compliance decision. It receives grouped and redacted issue metadata and produces human-friendly explanations or review guidance. If the API is unavailable, the system uses rule-based fallback guidance. This design keeps the core checker deterministic and makes the AI layer optional.

3.9 Evaluation Metrics

The evaluation framework includes:

- Smoke testing: verifies that the checker runs on realistic samples.
- Auto-fix before/after evaluation: compares issue counts and compliance index before and after correction.
- Post-fix safety validation: checks whether the corrected document increases detected issues.
- Precision: proportion of detected issues that are correct.
- Recall: proportion of expected issues that are detected.
- F1-score: harmonic mean of precision and recall.
- Subject expert evaluation: measures usefulness, clarity, time saving, and trust.

3.10 Summary

The theoretical framework supports a transparent and practical system design. Compliance checking is deterministic, journal differences are represented through profiles, and optional AI is limited to explanation.

Chapter 4: Research Methodology and Proposed Prototype

4.1 Introduction

This chapter describes the research design, prototype architecture, methods, tools, and evaluation plan used in the project.

4.2 Research Design

This project follows a research-based design supported by proof-of-concept prototype implementation. The research investigates whether a rule-first, profile-based approach can support academic manuscript template compliance checking. The prototype is used to demonstrate the approach and produce evaluation evidence.

4.3 Overall Research Framework

The proposed framework consists of six stages:

11. Template rule acquisition.
12. Journal profile loading and rule merging.
13. Manuscript feature extraction.
14. Rule-based paragraph classification.
15. Compliance checking and issue generation.
16. Safer auto-fix, reporting, and post-fix validation.

4.4 JIWE Profile Rule Representation

JIWE is used as the primary validated journal profile. The profile stores required sections, required declarations, classification patterns, rule weights, and formatting rules. Important rules include Times New Roman body text at 10 pt, justified alignment, JIWE reference formatting at 9 pt with 1.15 line spacing, abstract length of 200 to 300 words, and required declarations such as funding statement and data availability.

4.5 Template Rule Extraction Method

The system can extract rules from uploaded DOCX journal templates using `modules/template_extractor.py`. It reads document paragraphs, formatting properties, instruction text, word limits, keyword requirements, reference requirements, and other observable style information. The implementation uses `python-docx`, a Python library for reading and updating DOCX documents (python-docx, 2026). Extracted rules are combined with profile defaults and provenance information.

4.6 Manuscript Feature Extraction Method

The system uses `python-docx` to extract paragraph text, run formatting, alignment, line spacing, section margins, page size, header and footer information, and document

structure. Utility functions in `modules/utils.py` support font extraction, margin extraction, line spacing reading, reference detection, and compliance score calculation.

4.7 Rule-Based Paragraph Classification Method

Paragraph classification is performed by `modules/paragraph_classifier.py`. The classifier uses deterministic logic and profile patterns to identify document roles. It considers text content, regular expression patterns, formatting signals, and document position. This replaced the need for expensive LLM-first paragraph classification.

4.8 Compliance Checking Method

The main checker in `modules/manuscript_checker.py` applies rules to classified document elements. It produces issue records with category, description, current value, expected value, location, severity, paragraph index, and text preview. Issue categories include margins, layout, journal header, title, author information, body text, headings, structure, tables, figures, references, and other checks.

4.9 Auto-Correction Method

The auto-fix module applies deterministic corrections only for supported issue properties. It uses the detected issue list to avoid changing unrelated document content. After generating a corrected document, the system re-checks it and compares issue counts and compliance score before and after fixing. If new or increased issues appear, the system warns the user.

4.10 Report Generation Method

The report generator creates a comparison report that summarizes changed properties, target rules, detailed changes, compliance index, and remaining issues. The web interface also provides a corrected DOCX and highlighted original DOCX.

4.11 Prototype Architecture

The prototype architecture contains:

- Streamlit UI layer.
- Template extraction layer.
- Profile loading layer.
- Paragraph classification layer.
- Rule-based checking layer.
- Auto-fix layer.
- Report generation layer.
- Optional LLM explanation layer.
- Evaluation CLI layer.

Figure 4.1 should show this architecture as a data flow from template and manuscript inputs to checking, fixing, validation, and outputs.

4.12 Prototype Modules

The prototype modules are summarized below:

Module	Responsibility
app.py	User interface and workflow control
template_extractor.py	Template rule extraction
profile_loader.py	Profile loading and merging
paragraph_classifier.py	Paragraph role classification
manuscript_checker.py	Rule-based issue detection
auto_fixer.py	Supported formatting correction
report_generator.py	Comparison report output
review_guidance.py	Rule-based review guidance
llm_integration.py	Optional AI explanation
evaluate_checker.py	Evaluation and summary generation

4.13 Tools and Software

The prototype uses Python, Streamlit, python-docx, JSON profile files, optional pdf2docx for PDF upload conversion, optional NVIDIA API for explanations, and GitHub for version control. Streamlit is used to provide the web interface and file upload workflow (Streamlit, 2026). NVIDIA NIM-style API support is used only as an optional explanation service when a valid key is configured (NVIDIA, 2026). The app is designed so that the checker still works without an LLM API key.

4.14 Evaluation Design

The evaluation design has four parts:

17. Smoke evaluation using real JIWE samples.
18. Auto-fix before/after evaluation.
19. Manual-label evaluation for precision, recall, and F1-score.
20. Subject expert evaluation for usefulness, clarity, time saving, and trust.

At the current stage, smoke and auto-fix evaluations are available in `evaluation_results`. Manual-label and subject expert evaluation are presented as recommended validation steps because the available repository artifacts do not contain completed human-labelled accuracy results.

Table 4.2: Recommended Subject Expert Evaluation Items

Item	Measurement	Purpose
Usefulness	1 to 5 Likert rating	Measures whether the

		system helps initial manuscript screening
Clarity	1 to 5 Likert rating	Measures whether issue descriptions and guidance are understandable
Time saving	Manual time compared with system-assisted time	Measures practical efficiency
Trust	1 to 5 Likert rating	Measures whether reviewers trust the rule-based output
Remaining manual workload	Short comment	Identifies issues that still require human judgement
Suggested improvements	Short comment	Guides future enhancement

4.15 Summary

This chapter described the methods used to implement and evaluate the prototype. The methodology emphasizes deterministic validation, profile-based rules, safer auto-correction, and transparent evaluation.

Chapter 5: Prototype Implementation, Data Analysis and Results

5.1 Introduction

This chapter presents the implemented prototype features and available evaluation results.

5.2 Experimental Setup

The prototype was evaluated using JIWE template rules and sample manuscripts stored outside the repository. The evaluation CLI `evaluate_checker.py` was used to run smoke tests and auto-fix before/after evaluation. The checker processed DOCX files, detected formatting issues, applied supported fixes, re-checked corrected outputs, and generated summary artifacts.

5.3 Test Documents Used

Two main sample groups were used in the available evaluation artifacts:

- Published JIWE sample group: 10 documents.
- Declined or flawed sample group: 6 documents.

Invalid samples skipped in the reported runs: 0.

5.4 Implemented Prototype Features

The implemented prototype includes:

- Template rule upload and extraction.
- Default JIWE profile rules.
- Manuscript upload and checking.
- Compliance index display.
- Document structure summary.
- Issue category tabs.
- Optional issue explanation.
- Auto-fix button.
- Post-fix validation.
- Corrected DOCX download.
- Highlighted original DOCX download.
- Detailed comparison report download.

Additional regression-tested refinements were added during FYP2 development. The checker now detects keyword font mismatches, table caption font and italic mismatches, figure caption italic mismatches, JIWE main heading capitalization errors, and reference source italic requirements that should be reviewed manually. The auto-fix module also

supports safer correction of bulk body bold formatting, heading capitalization, and existing keyword capitalization. These refinements improve the system's ability to handle intentional formatting errors such as whole-document bold formatting, incorrect fonts, and title-case headings.

5.5 User Interface Results

The Streamlit interface presents the workflow in stages: upload template, upload manuscript, check format, review issues, run auto-fix, review post-fix validation, and download outputs. The sidebar shows the active profile and extracted rule summary. The interface also explains that AI explanations are optional and that core checking remains rule-based.

5.6 Formatting Issue Detection Results

The published sample evaluation detected formatting and structure issues across 10 documents. The declined or flawed sample evaluation detected a higher number of issues in some documents, especially in headings, references, and body text. These results show that the checker can identify multiple categories of compliance issues in realistic documents.

5.7 Rule Category Results

Common issue categories found in the available evaluation artifacts include body text, headings, references, structure, tables, figures, journal header, layout, and other document-level issues. The results support the need for category-based reporting because different issue types require different correction strategies.

5.8 Compliance Index Before and After Auto-Fix

For the published sample run:

- Documents checked: 10.
- Total issues before auto-fix: 88.
- Total issues after auto-fix: 14.
- Average compliance index before auto-fix: 91.64.
- Average compliance index after auto-fix: 97.34.
- Safe post-fix validations: 10 out of 10.

For the declined or flawed sample run:

- Documents checked: 6.
- Total issues before auto-fix: 98.
- Total issues after auto-fix: 28.
- Average compliance index before auto-fix: 87.62.
- Average compliance index after auto-fix: 97.47.

- Safe post-fix validations: 6 out of 6.

5.9 Auto-Fix Evaluation

The auto-fix evaluation shows that supported deterministic formatting fixes reduced detected issues in both sample groups. For published samples, total detected issues were reduced from 88 to 14. For declined or flawed samples, total detected issues were reduced from 98 to 28. The remaining issues generally represent cases that require manual review, such as missing structure, caption problems, or reference issues that cannot always be corrected safely without changing content meaning.

5.10 Post-Fix Validation Results

The post-fix validation mechanism re-checks the corrected document and compares it with the original result. In the available summary runs, all documents were marked safe because the post-fix result did not increase detected issues. This supports the design decision to use conservative issue-based auto-fix rather than broad document rewriting.

5.11 Regression Testing Results

Regression testing was used after implementation changes to confirm that existing functionality was not broken. The final automated test suite contains 103 unit and regression tests covering template extraction, profile loading, paragraph classification, issue detection, auto-fix behaviour, review guidance, and report generation. The latest verification run passed all 103 tests. Python compilation checks also passed, confirming that the source files have no syntax errors.

The most important intentional-error test changed a manuscript to use incorrect formatting across many paragraphs, including wrong font, oversized text, bold body text, and non-compliant headings. Before auto-fix, the system detected 46 issues across categories such as journal header, title, author information, body text, headings, references, and line spacing. After auto-fix, only three issues remained: abstract word count, uncited reference, and reference publication source italic review. These remaining items are appropriate manual-review cases because they involve content or reference interpretation rather than a simple deterministic formatting change.

5.12 Processing Time Evaluation Limitation

Processing time was not clearly available in the current evaluation result files. Therefore, the report does not claim a measured time-saving percentage. A future efficiency study should record the time taken per document for manual checking and system-assisted checking. This would support a stronger claim about practical screening efficiency.

5.13 Summary of Findings

The available results indicate that the prototype can process realistic JIWE manuscripts, detect multiple formatting issue categories, apply deterministic fixes, and improve compliance index values after auto-fix. However, formal detection accuracy still requires manually labelled ground truth and subject expert validation.

Chapter 6: Discussion

6.1 Introduction

This chapter interprets the results, discusses strengths and limitations, and explains the role of the proposed approach in manuscript screening.

6.2 Interpretation of Results

The results show that a rule-first approach can detect and reduce many objective formatting issues. The reduction of detected issues after auto-fix suggests that deterministic formatting correction is useful for repetitive and clearly defined problems. However, remaining issues after auto-fix show that not every issue should be corrected automatically.

6.3 Effectiveness of Rule-Based Validation

Rule-based validation is effective for objective formatting checks because expected values are explicit. Examples include font size, font name, bold, italic, alignment, margin size, line spacing, required section presence, and reference line spacing. The main advantage is explainability: each issue can show the current value and expected value.

6.4 Effectiveness of Profile-Based JIWE Rules

The JIWE profile allows the checker to apply journal-specific rules without embedding every rule directly in the checking code. This makes the approach more maintainable and allows future journal profiles to be added. However, each new journal profile must still be validated using real samples.

6.5 Role of Optional AI Explanation Layer

The optional AI layer improves usability by explaining detected issues in natural language. It does not perform core detection, correction, or approval. This design responds to concerns that large LLMs may be inefficient or unreliable for objective formatting checks. When the LLM is unavailable, deterministic fallback guidance is still provided.

6.6 Strengths of the Proposed Approach

The strengths of the prototype include:

- Explainable rule-based detection.
- Journal profile support.
- JIWE validated default profile.
- Conservative auto-fix.
- Corrected, highlighted, and report outputs.
- Post-fix validation.

- Optional AI explanation without LLM-first checking.
- Evaluation CLI for smoke tests and labelled metrics.

6.7 Limitations

The limitations include:

- DOCX is the primary reliable format; PDF conversion can lose layout fidelity.
- PDF output is not guaranteed on every server environment.
- Multi-journal support requires additional validated profiles.
- The compliance index is not formal accuracy.
- Manual labels are still needed for precision, recall, and F1-score.
- The system does not assess research quality.
- The system does not safely move figures or tables.
- Some complex reference formatting and citation verification remain manual.
- Reference publication source italics can be flagged, but automatic correction is left for manual review because different reference entries contain different source boundaries.
- PDF output is intentionally limited on the deployed server because LibreOffice conversion can shift Word layout; the safer workflow is to download DOCX and export to PDF using Microsoft Word.

6.8 Comparison with Existing Tools

Compared with manual checking, the prototype can reduce repetitive work and provide consistent issue categories. Compared with grammar tools, it focuses on journal formatting rather than language correctness. Compared with citation managers, it checks broader document structure and formatting. Compared with generic AI tools, it uses deterministic rules for compliance decisions and keeps AI optional.

6.9 Implications for Authors and Editors

For authors, the system can be used before submission to identify common formatting problems. For editors or lecturers, the system can support initial screening by producing a structured issue list and corrected document. Final judgement should still remain with human reviewers, especially for content quality, research contribution, and ambiguous layout issues.

6.10 Summary

The discussion shows that the proposed approach is practical when positioned as an initial screening and formatting assistance tool. Its strongest value is transparent rule-based checking combined with conservative auto-fix.

Chapter 7: Conclusion and Recommendations

7.1 Conclusion

This project proposed and implemented a rule-first, profile-based manuscript template compliance checker for academic journals. The prototype uses JIWE as the primary validated case study and demonstrates how deterministic rules can be used to detect objective formatting and structure issues. The system also supports safer auto-correction, highlighted outputs, comparison reports, post-fix validation, and optional AI explanations.

7.2 Achievement of Research Objectives

The project achieved the main objectives as follows:

21. A profile-based rule representation was implemented using JSON profiles.
22. A rule-based checker was implemented for JIWE manuscript compliance.
23. Safer auto-correction was implemented for supported deterministic formatting issues.
24. User-facing outputs were implemented, including corrected DOCX, highlighted DOCX, and comparison report.
25. Smoke and auto-fix before/after evaluation were conducted, while manual-label and subject expert evaluation remain recommended for final validation.

7.3 Research Contributions

The contributions of this project are:

- A rule-first architecture for manuscript template compliance checking.
- A validated JIWE profile for journal-specific rules.
- A conservative auto-fix approach with post-fix validation.
- A separation between deterministic compliance decisions and optional AI explanations.
- An evaluation workflow that supports smoke testing, auto-fix analysis, and labelled precision, recall, and F1-score evaluation.

7.4 Project Limitations

The project is limited by its reliance on DOCX as the primary editable format, partial PDF support, need for validated profiles for additional journals, lack of final manual-label metrics in the current artifacts, and the need for subject expert evaluation. The system also does not replace human editorial judgement.

7.5 Recommendations for Future Work

Future work should include:

26. Complete manual-label evaluation using at least 50 diverse manuscripts.

27. Conduct subject expert evaluation with lecturers, editors, or trained reviewers.
28. Add more validated journal profiles.
29. Improve reference formatting checks with safer rule-based patterns.
30. Explore lightweight layout-aware classifiers only if enough labelled data is collected.
31. Improve PDF handling through a staged fallback pipeline while clearly warning about fidelity limits.
32. Add a user correction interface for ambiguous paragraph classifications.

7.6 Final Summary

The project demonstrates that a rule-first, profile-based approach is a practical and explainable method for academic manuscript template compliance checking. The prototype does not attempt to replace reviewers or guarantee journal acceptance. Instead, it supports authors and reviewers by detecting common formatting issues, applying safe corrections, and presenting remaining issues for manual review.

References

- Pfitzmann, B., Auer, C., Dolfi, M., Nassar, A. S., & Staar, P. W. J. (2022). DocLayNet: A large human-annotated dataset for document-layout analysis. Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. <https://doi.org/10.1145/3534678.3539043>
- Lopez, P. (2009). GROBID: Combining automatic bibliographic data recognition and term extraction for scholarship publications. Proceedings of the 13th European Conference on Research and Advanced Technology for Digital Libraries. https://doi.org/10.1007/978-3-642-04346-8_62
- Microsoft. (2024). Create a word processing document by providing a file name. Microsoft Learn. <https://learn.microsoft.com/en-us/office/open-xml/word/how-to-create-a-word-processing-document-by-providing-a-file-name>
- NVIDIA. (2026). NVIDIA NIM for large language models API reference. NVIDIA Docs. <https://docs.nvidia.com/nim/large-language-models/latest/api-reference.html>
- python-docx. (2026). python-docx documentation. <https://python-docx.readthedocs.io/>
- Streamlit. (2026). st.file_uploader. Streamlit Docs. https://docs.streamlit.io/develop/api-reference/widgets/st.file_uploader
- Xu, Y., Li, M., Cui, L., Huang, S., Wei, F., & Zhou, M. (2020). LayoutLM: Pre-training of text and layout for document image understanding. Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. <https://doi.org/10.1145/3394486.3403172>
- Zhong, X., Tang, J., & Jimeno Yepes, A. (2019). PubLayNet: Largest dataset ever for document layout analysis. Proceedings of the International Conference on Document Analysis and Recognition. <https://arxiv.org/abs/1908.07836>

Appendices

Appendix A: Updated Gantt Chart

This appendix should include the final FYP2 project schedule and completed milestones.

Appendix B: FYP2 Meeting Logs

This appendix should include meeting dates, discussion points, and actions taken.

Appendix C: Turnitin Similarity Index Page

This appendix should include the Turnitin similarity page when available.

Appendix D: Evaluation Result Samples

This appendix should include selected outputs from `evaluation_results`, especially published and declined auto-fix summaries.

Appendix E: Subject Expert Evaluation Form

This appendix should include the subject expert rating form covering usefulness, clarity, time saving, trust, and remaining manual review.

Appendix F: Selected Prototype Screenshots

This appendix should include screenshots of upload, checking, issue categories, review guidance, auto-fix, and download outputs.

Appendix G: Code Repository Summary

This appendix should include the repository structure, major modules, test results, and GitHub commit evidence.